Course Name	Ethical Hacking
Course Code	ITA1409
Course Outcome	CO1: Apply the principle of the foundational concepts of ethical hacking, including types of hackers, ethical principles, legal frameworks, and the role of responsible disclosure in cybersecurity.
Student Name / Reg No	B Kirthanaa / 192324244

CO1 – AT3

Question:

You are conducting a penetration test and discover that the client's production database contains real customer financial data. Your test plan calls for exploiting a SQL injection vulnerability. How do you proceed without causing harm, and what precautions should have been taken before the engagement began?

Answer: Handling a SQL Injection Test on a Production Database with Real Customer Financial Data.

---

1. Understanding the Scenario

In this scenario, I am an ethical hacker (penetration tester) hired by a client. During the test, I find that the client's live production database holds real customer financial data. My test plan says I should exploit a SQL injection vulnerability, which is a flaw that lets an attacker send harmful database commands through an input field of an application. The question is how to continue the work without harming the customer or the client, and what should have been arranged before the test began.

Facts given in the question

- I am conducting a penetration test for a client.
- The client's production (live) database has real customer financial data.
- My test plan includes exploiting a SQL injection vulnerability.

Facts that are not given

The question does not say which database is used, how much data is stored, whether the contract allows testing on production, or whether a test copy of the system exists. So I have not assumed these things. The examples later in this answer are only general illustrations and not details of a real incident.

2. Why This Situation Is Risky

Customer financial data, such as account details or card information, is highly sensitive. If it is leaked, changed, or deleted, real people can suffer fraud, money loss, or privacy harm, and the client can face legal penalties. SQL injection is risky on a production system because a badly chosen action can read, change, or delete records, or even slow down or crash a service that customers are using at that moment.

A penetration tester is trusted with access. The core ethical rule is to cause no harm and to prove the weakness with the least possible impact. Finding a vulnerability is not a permission to take or show the data behind it.

3. How to Proceed Without Causing Harm

3.1 Pause and Reassess

The first step is to stop and think before running the planned exploit. I should ask whether the test plan was written knowing that real financial data is present, and whether the planned action can be done in a safer way. A plan is not a reason to continue if the risk has changed.

3.2 Check the Scope and Rules of Engagement

- Read the signed contract and Rules of Engagement (RoE) to see whether testing the production database is clearly allowed.
- If the document is unclear or does not mention live customer data, I should not guess. I should ask for written clarification first.
- Anything outside the written scope must not be touched.

3.3 Inform the Client and Get Direction

- Contact the agreed client contact right away, and explain that real financial data is exposed to the test.
- Ask whether the test should continue, move to a staging copy, or be run at a quiet time with the technical team ready.
- Keep the reply in writing, so that there is a record of who approved what.

3.4 Use Safe, Non-Destructive Proof

- The goal is only to prove the vulnerability exists, not to take the data. A harmless result, such as confirming that the input is handled unsafely, is enough as evidence.
- Use read-only techniques and avoid any action that can insert, change, or delete data.
- Do not try to view real customer records. If some data appears by accident, stop and do not go further.
- Keep the testing slow and light, and avoid anything that can overload the database or stop the service.

3.5 Handle Any Data Responsibly

- Follow the principle of data minimisation: collect only what is needed to prove the issue.
- Mask or hide sensitive values (for example, show only part of a number) in screenshots and notes.
- Never copy customer data to a personal device or share it over insecure channels. Evidence should be stored encrypted and deleted after the period agreed with the client.

3.6 Monitor, Stop, and Escalate

- Set clear stop conditions. If the system slows down, errors appear, or data seems to be affected, stop immediately and tell the client.
- If I find that data was already exposed or that someone else has attacked the system, I should report it at once as an incident.

3.7 Document and Report

- Record every step with date and time, so that the client can trace exactly what was done.
- Report a critical finding like this immediately and not only in the final report, because attackers could find the same flaw.
- Suggest fixes: parameterised queries (prepared statements), input validation, least-privilege database accounts, and encryption of sensitive fields.

4. Precautions That Should Have Been Taken Before the Engagement

Most of the risk in this scenario could have been reduced by good planning. The following precautions should have been completed before any testing started.

4.1 Written Authorization and Contract

- A signed contract and an authorization letter that give legal permission to test. Without written permission, the same actions could be treated as a crime.
- A Non-Disclosure Agreement (NDA) to protect everything the tester sees.

4.2 Clear Scope and Rules of Engagement

- A list of the systems, applications, and IP addresses that may be tested, and those that must not be tested.
- Allowed techniques, testing hours, and a clear statement about whether production systems and live data are included.

4.3 Data Handling Agreement

- Rules for how any sensitive data is seen, stored, protected, and destroyed, and who is responsible if something goes wrong.

- A plan to use masked or dummy data wherever possible.

4.4 A Safer Test Environment

- Where possible, test on a staging or test copy of the system with fake data, so that real customers are never at risk.
- If production testing is truly needed, choose a low-traffic time and limit what the tester can do.

4.5 Backups and Contingency Planning

- The client should take a recent, verified backup before testing starts, so that data can be restored if something breaks.
- A plan for incidents, with named emergency contacts on both sides who can be reached quickly, including outside normal hours.

4.6 Legal Compliance and Tester Readiness

- Check the laws and standards that apply to the data, such as the Information Technology Act, 2000 and the Digital Personal Data Protection Act, 2023 in India, and PCI DSS when card data is involved.
- Use trained, trusted testers, ideally with background checks, and professional indemnity or cyber insurance for the engagement.

5. Ethical and Legal Principles Involved

- **Authorization:** Testing is legal and ethical only with clear permission. Under the Information Technology Act, 2000 (for example Sections 43 and 66), unauthorized access or damage to a computer system is an offence.
- **Do no harm:** The tester must protect the confidentiality, integrity, and availability of the client's systems and of the customers' data.
- **Confidentiality:** Everything found during the test belongs to the client and must not be shared with anyone else.
- **Data protection:** Personal and financial data is protected by law, and mishandling it during a test can still create legal responsibility.
- **Responsible disclosure:** The vulnerability should be reported privately to the client with enough detail to fix it, and not made public or used for any other purpose.

6. Practical Example (General Illustration)

Note: This example is only to explain the idea. It is not a real incident and it is not part of the facts given in the question.

Imagine a tester who is hired to test an online banking portal. While checking the login search box, the tester notices signs of a SQL injection flaw and realizes that the live database holds real account details.

- **Unsafe way:** The tester pulls out customer records to show how serious the flaw is and saves them on a personal laptop. This exposes real people's data and breaks the contract and the law, even though the intention was good.
- **Safe way:** The tester stops, checks the Rules of Engagement, and informs the client. Together they agree to confirm the flaw using a harmless check, or to repeat it on a staging copy. The tester records only masked evidence, reports the issue at once, and helps the team fix it.

This shows that a good ethical hacker proves the risk without becoming the cause of the harm.

7. Comparison Table

The table below compares unsafe and safe ways of handling the main situations in this scenario.

Situation	Unsafe Approach	Safe Approach
Real financial data found	Browse, copy, or screenshot customer records to "prove" the flaw.	Stop, notify the client, and prove the flaw using a harmless, read-only check that does not show real records.
Testing method	Run heavy or destructive tests that can modify, delete, or lock data.	Use low-impact, read-only techniques in the agreed time window, and stop if the system slows down.
Data handling	Download data to a personal laptop or share it by email or chat.	Collect only the minimum evidence, mask the sensitive values, and store it encrypted for the agreed period.
Scope doubts	Carry on because "the test plan says so".	Pause and get written confirmation that the activity is within the scope and Rules of Engagement.
Reporting	Wait until the final report to mention the serious flaw.	Report critical findings to the client immediately, then give full details in the final report.
Before the engagement	Start testing with only a verbal approval.	Have a signed contract, authorization letter, NDA, and data handling terms in place first.

8. Conclusion

When a penetration test reaches real customer financial data, the correct approach is to pause, confirm the scope in writing, inform the client, and prove the SQL injection vulnerability in

the least harmful way, without viewing, copying, or changing customer records. Any data seen must be minimised, protected, and later destroyed, and the finding must be reported quickly with clear advice on how to fix it.

Most of this risk should have been handled before the engagement through signed authorization, a clear scope and Rules of Engagement, data handling terms, a test environment with dummy data where possible, verified backups, and legal compliance checks. Ethical hacking depends on permission, responsibility, and trust, and following these steps protects the customers, the client, and the tester.